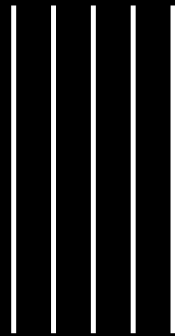


Git & GitHub Team Workflow



Afrizal F.A

Git & GitHub Team Workflow

Latihan tim 1–4 akun: Pull Request, review, CI, conflict, squash merge, hotfix, parallel work, dan cherry-pick

Afrizal F.A

2026

Kata Pengantar

Buku ini merupakan panduan praktik Git dan GitHub dalam kerja tim. Pembahasan mengikuti satu proyek dari persiapan repository, pembuatan branch, Pull Request, review, CI, penyelesaian conflict, hingga cherry-pick dan recovery.

Ikuti setiap bab secara berurutan karena seluruh latihan saling berhubungan. Gunakan perintah dan contoh file yang tersedia, lalu pastikan setiap tahap berhasil sebelum melanjutkan ke tahap berikutnya.

Daftar Isi

BAB 1 – Pendahuluan

BAB 2 – Memahami VCS, Git, dan GitHub

BAB 3 – Persiapan: SSH Multi-Akun dan Repository Awal

BAB 4 – Task Pertama: APP-101

BAB 5 – Review, CI, dan Revisi

BAB 6 – Konflik dan Integrasi

BAB 7 – Squash Merge, Tag Rilis, dan Hotfix

BAB 8 – Simulasi 3 Akun: Parallel Work

BAB 9 – Simulasi 4 Akun: Dua Reviewer

BAB 10 – Cherry-pick

BAB 11 – Error Handling dan Recovery

BAB 12 – Penutup

BAB 1 – Pendahuluan

Kamu bergabung ke tim fiksi Acme App Team yang mengelola repository `acme-profile-app`: aplikasi profil sederhana berbasis JavaScript. Lewat repository ini kamu menjalani siklus kerja tim software modern dari awal sampai selesai – dan repository yang SAMA dipakai terus dari Bab 3 sampai Bab 11.

```
main <- Pull Request <- feature branch <- commits lokal
```

Empat mode latihan

Mode	Akun	Yang dilatih
Pemanasan	1	Branch, commit, push, PR, merge
Realistis	2	Protected main, request changes, approval
Tim asli	3	Parallel work, review silang, konflik alami
Perusahaan	4	Dua reviewer, CODEOWNERS, approval ganda

Pembagian role

Akun	Role	Tanggung jawab
A	Tech Lead / Maintainer	Repo, aturan main, ticket, review akhir, merge
B	Developer 1 / Reviewer 1	Coding, PR, me-review PR orang lain
C	Developer 2 / Reviewer 2	Coding paralel, review silang (mode 3-4)
D	Developer (author)	Mengerjakan task, menangani feedback (mode 4)

Alur cerita buku ini (peta kesinambungan)

Bab	Kejadian	Jejak yang dibawa terus
3	Repo lahir (profile v0)	package.json, CI, README
4-5	APP-101 + review fix	validasi displayName
6	Main bergerak -> conflict	field updatedAt
7	Squash merge + tag + hotfix	v1.0.0, MAX_NAME 50
8	APP-103/104 paralel	location, formatProfile
9	APP-105, 2 reviewer	empty bio fix
10	release/1.0 dari tag	cherry-pick hotfix

Bab	Kejadian	Jejak yang dibawa terus
11	Error handling	semua via branch latihan/*

Prinsip utama buku ini: developer tidak pernah bekerja langsung di main. Semua perubahan masuk lewat branch dan Pull Request yang direview.

BAB 2 – Memahami VCS, Git, dan GitHub

Sebelum praktik lebih jauh, luruskan tiga istilah yang sering tertukar: VCS, Git, dan GitHub.

Apa itu VCS (Version Control System)

VCS mencatat setiap versi source code: apa yang berubah, siapa yang mengubah, kapan, dan kenapa (lewat pesan commit). Dengan VCS kamu bisa kembali ke versi mana pun, bekerja paralel lewat branch, dan menggabungkan pekerjaan banyak orang tanpa saling menimpa.

Jenis	Contoh	Ciri
Centralized (CVCS)	SVN, Perforce	Satu server pusat; sebagian besar operasi butuh online
Distributed (DVCS)	Git, Mercurial	Setiap clone = salinan penuh history; commit/branch/log jalan offline

Git vs GitHub

	Git	GitHub
Apa	VCS terdistribusi (program di komputermu)	Layanan hosting repo Git + kolaborasi
Kerjanya	commit, branch, merge, diff, log	PR, review, issues, Actions/CI, protection
Alternatif	Mercurial, Fossil	GitLab, Bitbucket, Codeberg

Karena itu "Pull Request" bukan perintah Git – itu fitur GitHub (di GitLab namanya Merge Request). Tanpa GitHub pun Git tetap jalan; GitHub-lah yang membuat kolaborasi tim nyaman.

Empat area kerja Git

```
working dir --add--> staging --commit--> local repo --push--> remote
               <--restore-      <--reset-              <--fetch/pull-
```

Area	Isi
Working directory	File yang sedang kamu edit
Staging (index)	Perubahan terpilih untuk commit berikutnya
Local repository	.git/ – seluruh history di komputermu
Remote (origin)	Salinan di GitHub yang dilihat semua orang

Istilah inti

Istilah	Makna praktis
commit	Snapshot ber-SHA + pesan; satuan terkecil history
branch	Pointer bergerak ke commit; jalur kerja paralel
HEAD	Posisimu sekarang (branch/commit aktif)
tag	Penanda permanen satu commit (mis. v1.0.0)
origin	Nama default remote hasil clone
fork	Salinan repo orang lain ke akunmu (fitur GitHub)

Fitur GitHub yang dipakai buku ini

Fitur	Fungsi	Dipakai di
Issues	Ticket/task (APP-101 dst.)	Bab 4
Pull Request	Usulan perubahan + diskusi review	Bab 4-10
Actions	CI otomatis (npm test)	Bab 3, 5
Branch protection	PR + approval + checks wajib	Bab 3, 9
CODEOWNERS	Reviewer otomatis per path	Bab 9
Releases + tag	Menandai versi rilis	Bab 7, 10
Fork	Kontribusi tanpa akses tulis	bab ini

Fork vs clone (alur open source)

Di dalam tim (punya akses tulis) kamu cukup clone. Di project orang lain (tanpa akses tulis) alurnya lewat fork:

```
Fork di GitHub -> clone fork-mu -> branch -> commit -> push
-> buka PR dari fork ke repo asal (upstream)

# jaga fork tetap sinkron dengan repo asal:
git remote add upstream acme-lead:<user-A>/acme-profile-app.git
git fetch upstream
git switch main && git merge upstream/main && git push
```

Tag dan Release

```
# tandai versi setelah rilis siap di main
git tag -a v1.0.0 -m "release 1.0.0"
git push origin v1.0.0
git tag -l          # daftar tag

# di GitHub: Releases -> Draft a new release -> pilih v1.0.0
```

Ingat konsep ini: di Bab 7 kamu akan MEMBUAT tag v1.0.0 tepat setelah APP-101 masuk main, dan di Bab 10 tag itu menjadi titik lahir release/1.0 – dasar simulasi cherry-pick.

BAB 3 – Persiapan: SSH Multi-Akun dan Repository Awal

Setup Git dasar

```
git --version
git config --global user.name "Nama Kamu"
git config --global user.email "email-kamu"
```

Multi-akun di satu PC lewat SSH

Untuk mode 2-4 akun di satu komputer: satu SSH key per akun, lalu daftarkan alias per akun di ~/.ssh/config. Nama di bawah ini contoh fiksi – ganti dengan nama akun latihanmu sendiri.

```
# 1) Buat satu key per akun (email = email akun GitHub tsb)
ssh-keygen -t ed25519 -C "lead@example.com" \
-f ~/.ssh/id_ed25519_acme_lead
ssh-keygen -t ed25519 -C "dev1@example.com" \
-f ~/.ssh/id_ed25519_acme_dev1
ssh-keygen -t ed25519 -C "dev2@example.com" \
-f ~/.ssh/id_ed25519_acme_dev2
ssh-keygen -t ed25519 -C "dev3@example.com" \
-f ~/.ssh/id_ed25519_acme_dev3
```

```
# 2) Daftarkan alias per akun (copy-paste utuh)
cat >> ~/.ssh/config << 'EOF'
Host acme-lead
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_acme_lead
  IdentitiesOnly yes

Host acme-dev1
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_acme_dev1
  IdentitiesOnly yes

Host acme-dev2
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_acme_dev2
  IdentitiesOnly yes

Host acme-dev3
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_ed25519_acme_dev3
  IdentitiesOnly yes
EOF
```

```
# 3) Upload public key ke akun GitHub masing-masing
# (Settings -> SSH and GPG keys -> New SSH key)
cat ~/.ssh/id_ed25519_acme_lead.pub
cat ~/.ssh/id_ed25519_acme_dev1.pub
cat ~/.ssh/id_ed25519_acme_dev2.pub
cat ~/.ssh/id_ed25519_acme_dev3.pub

# 4) Test: harus menyapa username akun yang sesuai
ssh -T acme-lead
ssh -T acme-dev1

# (opsional) simpan passphrase agar tidak ditanya terus
ssh-add ~/.ssh/id_ed25519_acme_dev1
```

Aturan pakainya: saat clone/remote, GANTI git@github.com dengan alias akunnya. Satu folder clone = satu akun = satu identitas.

```
# clone per akun via alias (folder terpisah per role)
git clone acme-lead:<user-A>/acme-profile-app.git team-work-a
git clone acme-dev1:<user-A>/acme-profile-app.git team-work-b
git clone acme-dev2:<user-A>/acme-profile-app.git team-work-c
git clone acme-dev3:<user-A>/acme-profile-app.git team-work-d

# set identitas commit PER folder (tanpa --global)
cd team-work-b
git config user.name "Acme Dev One"
git config user.email "dev1@example.com"

# repo terlanjur di-clone pakai github.com? ganti remote-nya:
git remote set-url origin acme-dev1:<user-A>/acme-profile-app.git
git remote -v
```

Project awal (Akun A – Tech Lead)

```
WEB UI – buat repository (Akun A):
1. Login akun A -> klik "+" (kanan atas) -> "New repository"
2. Owner: <user-A>; Repository name: acme-profile-app
3. Visibility: Private (atau Public, bebas)
4. JANGAN centang "Add a README" – repo harus kosong
5. Klik "Create repository"
```

Repo kosong sudah ada. Sekarang buat file-file project. Ada DUA cara – hasilnya identik, pilih yang kamu suka:

Cara 1 – lewat terminal (heredoc, tinggal copy-paste)

Copy-paste blok cat > file << 'EOF' ... EOF apa adanya; file langsung jadi lengkap dengan isinya.

Cara 2 – lewat text editor (nano / VS Code)

```
nano src/profile.js      # atau: code src/profile.js
# paste HANYA isi di antara baris cat ... << 'EOF' dan
# baris EOF (dua baris itu BUKAN bagian file), lalu simpan:
# nano : Ctrl+O, Enter, lalu Ctrl+X
# VS Code: Cmd+S (mac) / Ctrl+S
```

Contoh: bila memakai editor, isi src/profile.js versi awal adalah persis ini (tanpa baris cat/EOF):

```
// src/profile.js
function createProfile(displayName, bio) {
  return { displayName, bio };
}

module.exports = { createProfile };
```

Konvensi indentasi (penting!)

Jenis file	Indentasi	Alasan
JavaScript, JSON	TAB (\t)	Konsisten; JS/JSON bebas whitespace
YAML (ci.yml)	SPASI (wajib)	Spec YAML MELARANG tab
~/.ssh/config, README	bebas (buku: spasi)	Tidak sensitif

Jadi semua blok JS/JSON di buku ini ber-indentasi TAB. Kalau editormu mengubah tab menjadi spasi (auto-convert), tidak masalah untuk JS/JSON – yang FATAL hanya mengubah spasi YAML menjadi tab.

```
mkdir acme-profile-app && cd acme-profile-app
mkdir -p src test .github/workflows
```

```
cat > package.json << 'EOF'
{
  "name": "acme-profile-app",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "test": "node --test"
  }
}
EOF
```

package.json ini WAJIB. Tanpa script "test": "node --test", perintah npm test akan gagal dengan pesan "Error: no test specified".

```
cat > src/profile.js << 'EOF'
// src/profile.js
function createProfile(displayName, bio) {
  return { displayName, bio };
}

module.exports = { createProfile };
EOF
```

```
cat > test/profile.test.js << 'EOF'
// test/profile.test.js (node:test, tanpa dependency)
const { test } = require("node:test");
const assert = require("node:assert/strict");
const { createProfile } = require("../src/profile");

test("creates a profile", () => {
  assert.equal(createProfile("Raka", "dev").displayName, "Raka");
});
EOF
```

```
cat > .github/workflows/ci.yml << 'EOF'
# YAML: indentasi WAJIB spasi – JANGAN pakai tab di file ini
name: ci

on:
  pull_request:
  push:
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm test
EOF
```

YAML sensitif terhadap indentasi DAN menolak tab – inilah satu-satunya file yang wajib spasi. Kalau kamu mengetik ulang dan spasinya bergeser (atau editor mengubahnya jadi tab), CI akan gagal parse – selalu pakai blok copy-paste di atas.

```
cat > .gitignore << 'EOF'
node_modules/
EOF
```



```
cat > README.md << 'EOF'
# acme-profile-app
```

Project latihan kolaborasi Git & GitHub (Acme App Team).

```
## Testing
```

```
    npm test
EOF
```

```
# cek cepat hasil paste: sintaks + spasi harus utuh
node --check src/profile.js
node --check test/profile.test.js
cat .github/workflows/ci.yml # indentasi harus persis
```

```
# pastikan test lulus SEBELUM commit pertama
npm test
```

```
git init
git add .
git commit -m "chore: initialize profile app"
git branch -M main
git remote add origin acme-lead:<user-A>/acme-profile-app.git
git push -u origin main
```

Aturan branch main

Aturan	Tujuan
Require Pull Request	Tidak ada direct push ke main
Require approval	Minimal 1 reviewer setuju (mode 4: minimal 2)
Require status checks	CI harus hijau sebelum merge
Block force push	History main tidak ditulis ulang

WEB UI – tambah collaborator (Akun A):

1. Repo -> Settings -> Collaborators -> "Add people"
2. Cari username akun B -> Add (ulangi utk C, D sesuai mode)
3. Akun B/C/D buka notifikasi/email -> "Accept invitation"

WEB UI – branch protection (Akun A):

1. Repo -> Settings -> Branches -> "Add branch ruleset" (di UI lama: "Add rule"); target branch: main
2. Centang "Require a pull request before merging" -> Required approvals: 1 (di Bab 9 dinaikkan jadi 2)
3. Centang "Require status checks to pass" -> pilih check "test" (muncul setelah CI pernah jalan 1x)
4. Centang "Block force pushes" -> Save

Jika fitur protection tidak tersedia di plan repositorymu, jalankan aturannya secara disiplin manual.

BAB 4 – Task Pertama: APP-101

Ticket dari Tech Lead

```
APP-101 – Add editable user profile
- displayName dan bio tersimpan
- displayName wajib diisi
- Spasi depan/belakang dibersihkan
- Test lulus
Out of scope: avatar, database, auth
```

```
WEB UI – buat Issue (Akun A):
1. Repo -> tab "Issues" -> "New issue"
2. Title: APP-101 – Add editable user profile
3. Body: salin isi ticket di atas
4. Panel kanan: Assignees -> pilih akun B
5. "Submit new issue" -> catat nomornya (misal #1);
   tulisan "Closes #1" di PR akan menutupnya otomatis
```

Urutan yang benar: branch DULU, baru coding

Kesalahan paling umum: coding dan commit dulu di main, lalu push feature branch yang belum ada – hasilnya error "src refspec ... does not match any". Selalu buat branch sebelum menyentuh code, dan cek posisi branch sebelum setiap commit.

```
# Akun B, di folder team-work-b
git switch main
git pull origin main
git switch -c feature/APP-101-editable-profile

# WAJIB sebelum commit apa pun – harus tercetak nama feature branch
git branch --show-current
# feature/APP-101-editable-profile
```

Kalau terlanjur salah branch

```
# terlanjur EDIT di main (belum commit): bawa perubahan pindah
git switch -c feature/APP-101-editable-profile

# terlanjur COMMIT di main (belum push): pindahkan commit
git branch feature/APP-101-editable-profile
git reset --hard origin/main
git switch feature/APP-101-editable-profile
```

Commit 1 – normalisasi (file UTAH, copy-paste)

```

cat > src/profile.js << 'EOF'
// src/profile.js
function createProfile(displayName, bio = "") {
  return {
    displayName: displayName.trim(),
    bio: bio.trim(),
  };
}

module.exports = { createProfile };
EOF

```

Perhatikan: baris `module.exports` SELALU ada di akhir file. Kalau hilang, test langsung gagal dengan "createProfile is not a function".

```

git branch --show-current    # feature/APP-101-editable-profile
git status && git diff
npm test                    # semua PASS dulu, baru commit
git add src/profile.js
git commit -m "feat: normalize profile fields"

```

Commit 2 – validation + test (file UTUH, copy-paste)

```

cat > src/profile.js << 'EOF'
// src/profile.js
function createProfile(displayName, bio = "") {
  const cleanName = displayName.trim();
  const cleanBio = bio.trim();

  if (!cleanName) {
    throw new Error("displayName is required");
  }

  return { displayName: cleanName, bio: cleanBio };
}

module.exports = { createProfile };
EOF

```

```

cat > test/profile.test.js << 'EOF'
// test/profile.test.js
const { test } = require("node:test");
const assert = require("node:assert/strict");
const { createProfile } = require("../src/profile");

test("creates a profile", () => {
  assert.equal(createProfile("Raka", "dev").displayName, "Raka");
});

test("trims whitespace", () => {
  assert.equal(createProfile("  Raka  ").displayName, "Raka");
});

test("rejects empty displayName", () => {
  assert.throws(() => createProfile("  "), /required/);
});
EOF

```

```

git branch --show-current    # feature/APP-101-editable-profile
npm test
git add src/profile.js test/profile.test.js
git commit -m "feat: validate required display name"

```

Push dan buka Pull Request

```
git push -u origin feature/APP-101-editable-profile
```

WEB UI – buka PR (Akun B):

1. Buka repo -> muncul banner kuning "feature/APP-101-... had recent pushes" -> klik "Compare & pull request" (alternatif: tab "Pull requests" -> "New pull request" -> base: main <- compare: feature/APP-101-...)
2. Isi Title + description dengan template di bawah
3. Panel kanan: Reviewers -> pilih Akun A
4. Klik "Create pull request" -> jadilah PR #14

```

Title   : APP-101: add editable user profile validation
Summary : normalize + validate displayName/bio, add tests
Test    : npm test; " Raka " -> "Raka"; nama kosong -> error
Closes #1 (ticket APP-101)

```

Sebelum minta review, lakukan self-review di tab Files changed: cari debug code, file nyasar, secret, atau perubahan di luar scope.

Error umum di fase awal (dan obatnya)

Error	Penyebab	Solusi
Error: no test specified	package.json tidak ada / script test kosong	Buat package.json persis seperti Bab 3

Error	Penyebab	Solusi
<code>createProfile</code> is not a function	<code>module.exports</code> terhapus saat edit	Tulis file utuh dari blok copy-paste
<code>displayName</code> is not defined	Menyalin potongan code, bukan file utuh	Selalu pakai versi file lengkap
<code>src refspec ... does not match any</code>	Branch belum dibuat; commit nyasar di main	<code>git switch -c dulu</code> ; cek <code>git branch --show-current</code>
CI gagal parse YAML	Indentasi rusak saat mengetik ulang	Copy-paste blok <code>ci.yml</code> apa adanya

BAB 5 – Review, CI, dan Revisi

Checks otomatis

```
Pull Request #14
[PASS] ci / test
[WAIT] review
```

CI merah? Jangan minta reviewer mengabaikannya. Baca log, perbaiki di branch yang sama, push – PR terupdate otomatis.

Request changes (Akun A)

```
WEB UI – memberi review (Akun A):
1. Buka PR #14 -> tab "Files changed"
2. Arahkan kursor ke baris displayName.trim() -> klik "+"
3. Tulis komentar -> "Start a review"
4. Klik "Review changes" (kanan atas)
   -> pilih "Request changes" -> "Submit review"
```

```
src/profile.js
"displayName.trim() error bila input bukan string.
Tangani input invalid secara eksplisit + test."
```

```
Review result: REQUEST CHANGES
```

Request changes itu normal, bukan kegagalan. Jangan buat PR baru – perbaiki di branch yang sama.

Menangani review (Akun B) – file UTUH, copy-paste

```
git switch feature/APP-101-editable-profile
git branch --show-current # pastikan benar
```

```
cat > src/profile.js << 'EOF'
// src/profile.js
function createProfile(displayName, bio = "") {
  if (typeof displayName !== "string") {
    throw new Error("displayName must be a string");
  }

  const cleanName = displayName.trim();
  const cleanBio = typeof bio === "string" ? bio.trim() : "";

  if (!cleanName) {
    throw new Error("displayName is required");
  }

  return { displayName: cleanName, bio: cleanBio };
}

module.exports = { createProfile };
EOF
```



```

cat > test/profile.test.js << 'EOF'
// test/profile.test.js
const { test } = require("node:test");
const assert = require("node:assert/strict");
const { createProfile } = require("../src/profile");

test("creates a profile", () => {
  assert.equal(createProfile("Raka", "dev").displayName, "Raka");
});

test("trims whitespace", () => {
  assert.equal(createProfile("  Raka  ").displayName, "Raka");
});

test("rejects empty displayName", () => {
  assert.throws(() => createProfile("  "), /required/);
});

test("rejects non-string displayName", () => {
  assert.throws(() => createProfile(123), /must be a string/);
  assert.throws(() => createProfile(null), /must be a string/);
});
EOF

```

```

npm test
git add src/profile.js test/profile.test.js
git commit -m "fix: handle invalid profile input"
git push

```

WEB UI – setelah push perbaikan (Akun B):

1. Di thread komentar PR: tulis balasan singkat, misal "Handled invalid input + added coverage"
2. Klik "Resolve conversation" pada thread tsb
3. Panel Reviewers -> klik ikon panah berputar di samping nama A (re-request review)

A belum langsung approve – di bab berikutnya main bergerak duluan, dan PR ini harus integrasi dulu.

BAB 6 – Konflik dan Integrasi

Simulasi: main bergerak saat PR masih direview

Sekarang kondisi nyata: sementara PR APP-101 milik B masih menunggu re-review, Akun A memasukkan perubahan lain ke main – menambah field updatedAt. Baris yang diubah A bersinggungan dengan baris yang diubah B: resep conflict yang kita inginkan.

```
# Akun A - di folder team-work-a
git switch main && git pull origin main
```

```
cat > src/profile.js << 'EOF'
// src/profile.js
function createProfile(displayName, bio) {
  return { displayName, bio, updatedAt: Date.now() };
}

module.exports = { createProfile };
EOF
```

```
npm test
git commit -am "feat: add updatedAt timestamp"
git push origin main
# (A admin; untuk latihan boleh bypass protection,
# atau lakukan lewat PR kilat)
```

```
A---B---E main          (maju: updatedAt)
      \--C---D feature/APP-101 (tertinggal)
```

Di halaman PR #14 kini muncul peringatan "This branch has conflicts that must be resolved" dengan tombol Resolve conflicts. JANGAN pakai editor web untuk latihan ini – selesaikan lewat terminal, karena kasus nyata sering terlalu rumit untuk editor web.

Akun B: ambil info remote

```
# Akun B - di folder team-work-b
git fetch origin
git log --oneline --graph --decorate --all
```

Merge vs rebase

Cara	Efek	Catatan
git merge origin/main	History asli + merge commit	Aman untuk branch yang sudah dipush
git rebase origin/main	History linear, commit ditulis ulang	Butuh force push; ikuti policy tim

Praktik: merge origin/main (conflict yang diharapkan)

```
git switch feature/APP-101-editable-profile
git merge origin/main
# Auto-merging src/profile.js
# CONFLICT (content): Merge conflict in src/profile.js
```

```
<<<<<< HEAD
    return { displayName: cleanName, bio: cleanBio };
=====
    return { displayName, bio, updatedAt: Date.now() };
>>>>>> origin/main
```

- Saat MERGE: HEAD = branch feature-mu; sisi bawah = origin/main.
- Saat REBASE: terbalik - HEAD = main (base baru), sisi bawah = commit feature-mu yang sedang direplay.
- Jangan asal hapus marker: maksud A (updatedAt) dan maksud B (validasi + trim) harus sama-sama selamat, dan file kembali UTUH.

Resolusi (file UTUH, copy-paste)

```
cat > src/profile.js << 'EOF'
// src/profile.js
function createProfile(displayName, bio = "") {
  if (typeof displayName !== "string") {
    throw new Error("displayName must be a string");
  }

  const cleanName = displayName.trim();
  const cleanBio = typeof bio === "string" ? bio.trim() : "";

  if (!cleanName) {
    throw new Error("displayName is required");
  }

  return { displayName: cleanName, bio: cleanBio,
    updatedAt: Date.now() };
}

module.exports = { createProfile };
EOF
```

```
npm test
git add src/profile.js
git commit          # merge commit
git push
```

Field updatedAt milik A dan validasi milik B kini hidup bersama. Mulai titik ini SEMUA versi src/profile.js di bab-bab berikutnya

membawa updatedAt – perhatikan kesinambungannya.

Alternatif rebase: `git rebase origin/main` memunculkan conflict yang sama dengan arah HEAD terbalik; selesaikan lalu `git rebase --continue` dan `push --force-with-lease --force-if-includes`.

Praktik rebase conflict yang aman (tanpa menyentuh branch tim) ada di Bab 11 Case 2.

Jangan pernah `git push --force` polos. `--force-with-lease` menolak push bila remote berubah tanpa sepengetahuanmu.

BAB 7 – Squash Merge, Tag Rilis, dan Hotfix

Squash and merge (Akun A)

```
feature: 3 commit + 1 merge commit resolusi conflict
main    : 1 commit ->
"APP-101: add editable user profile validation (#14)"
```

Syarat merge: checks hijau, approval lengkap, conversations resolved, conflict selesai.

WEB UI – approve + squash merge (Akun A):

1. Buka PR #14 -> tab "Files changed" -> "Review changes"
-> "Approve" -> "Submit review"
2. Tab "Conversation" -> pastikan tertulis:
"All checks have passed" + "Changes approved"
3. Klik dropdown pada tombol merge -> "Squash and merge"
4. Rapiakan pesan: "APP-101: add editable user profile validation (#14)" -> "Confirm squash and merge"
5. Klik "Delete branch"
6. Cek tab Issues: #1 tertutup otomatis (efek "Closes #1")

Setelah merge (Akun B)

```
git switch main && git pull origin main
git branch -d feature/APP-101-editable-profile
git push origin --delete feature/APP-101-editable-profile
```

Tandai rilis: tag v1.0.0 (janji dari Bab 2)

```
# Akun A – main kini berisi APP-101 (termasuk updatedAt)
git switch main && git pull origin main
git tag -a v1.0.0 -m "release 1.0.0: editable profile"
git push origin v1.0.0
git tag -l
```

WEB UI – GitHub Release (opsional):

1. Repo -> "Releases" (kolom kanan) -> "Draft a new release"
2. "Choose a tag" -> pilih v1.0.0
3. Title: "v1.0.0 – editable profile" -> "Publish release"

PENTING untuk kesinambungan: tag v1.0.0 dibuat SEBELUM hotfix APP-102 di bawah. Di Bab 10, release/1.0 lahir dari tag ini – karena itu release/1.0 TIDAK berisi APP-102, dan di situlah cherry-pick berperan.

Hotfix APP-102 – batasi displayName 50 karakter

```
# Akun B
git switch main && git pull origin main
git switch -c fix/APP-102-display-name-length
git branch --show-current # fix/APP-102-display-name-length
```

```

cat > src/profile.js << 'EOF'
// src/profile.js
const MAX_NAME = 50;

function createProfile(displayName, bio = "") {
  if (typeof displayName !== "string") {
    throw new Error("displayName must be a string");
  }

  const cleanName = displayName.trim();
  const cleanBio = typeof bio === "string" ? bio.trim() : "";

  if (!cleanName) {
    throw new Error("displayName is required");
  }
  if (cleanName.length > MAX_NAME) {
    throw new Error("displayName max 50 characters");
  }

  return { displayName: cleanName, bio: cleanBio,
    updatedAt: Date.now() };
}

module.exports = { createProfile };
EOF

```

```

cat > test/profile.test.js << 'EOF'
// test/profile.test.js
const { test } = require("node:test");
const assert = require("node:assert/strict");
const { createProfile } = require("../src/profile");

test("creates a profile", () => {
  assert.equal(createProfile("Raka", "dev").displayName, "Raka");
});

test("trims whitespace", () => {
  assert.equal(createProfile("  Raka  ").displayName, "Raka");
});

test("rejects empty displayName", () => {
  assert.throws(() => createProfile("  "), /required/);
});

test("rejects non-string displayName", () => {
  assert.throws(() => createProfile(123), /must be a string/);
});

test("boundary 50 ok, 51 ditolak", () => {
  assert.ok(createProfile("a".repeat(50)));
  assert.throws(() => createProfile("a".repeat(51)), /max 50/);
});
EOF

```



```
npm test
git add src/profile.js test/profile.test.js
git commit -m "fix: limit profile display name length"
git push -u origin fix/APP-102-display-name-length
```

WEB UI: alurnya sama dengan PR #14 -
"Compare & pull request" -> Reviewers: A -> approve ->
"Squash and merge" -> "Delete branch". Hasilnya commit #15.

Buka PR terpisah; setelah approve, A squash merge sebagai "fix: limit profile display name length (#15)". Hotfix harus kecil, satu scope, mudah direvert. Cek boundary: 50 diterima, 51 ditolak. Catat: commit #15 inilah yang akan di-cherry-pick di Bab 10.

BAB 8 – Simulasi 3 Akun: Parallel Work

Mode ini melatih hal yang tidak bisa disimulasikan 2 akun: dua developer bekerja bersamaan di file yang sama, review silang, dan konflik yang muncul alami. Titik awal: main sudah berisi APP-101 + updatedAt (Bab 6) + APP-102 (Bab 7).

Setup (Akun A)

- Tambahkan B dan C sebagai collaborator; protection: PR + 1 approval + CI.
- Buat dua ticket yang sengaja menyentuh src/profile.js:

```
APP-103 (B): tambah field opsional "location" (trim, default "")
APP-104 (C): tambah formatProfile(profile) -> "name - bio"
```

Akun B - APP-103 (file UTUH, copy-paste)

```
# di folder team-work-b
git switch main && git pull origin main
git switch -c feature/APP-103-location-field
git branch --show-current
```

```

cat > src/profile.js << 'EOF'
// src/profile.js
const MAX_NAME = 50;

function createProfile(displayName, bio = "", location = "") {
  if (typeof displayName !== "string") {
    throw new Error("displayName must be a string");
  }
  if (typeof location !== "string") {
    throw new Error("location must be a string");
  }

  const cleanName = displayName.trim();
  const cleanBio = typeof bio === "string" ? bio.trim() : "";
  const cleanLocation = location.trim();

  if (!cleanName) {
    throw new Error("displayName is required");
  }
  if (cleanName.length > MAX_NAME) {
    throw new Error("displayName max 50 characters");
  }

  return {
    displayName: cleanName,
    bio: cleanBio,
    location: cleanLocation,
    updatedAt: Date.now(),
  };
}

module.exports = { createProfile };
EOF

```

```

npm test
git add src/profile.js
git commit -m "feat: add optional location field"
git push -u origin feature/APP-103-location-field

```

Akun C – APP-104, dikerjakan BERSAMAAN (file UTUH)

```

# di folder team-work-c
git switch main && git pull origin main
git switch -c feature/APP-104-profile-formatter
git branch --show-current

```

```

cat > src/profile.js << 'EOF'
// src/profile.js
const MAX_NAME = 50;

function createProfile(displayName, bio = "") {
  if (typeof displayName !== "string") {
    throw new Error("displayName must be a string");
  }

  const cleanName = displayName.trim();
  const cleanBio = typeof bio === "string" ? bio.trim() : "";

  if (!cleanName) {
    throw new Error("displayName is required");
  }
  if (cleanName.length > MAX_NAME) {
    throw new Error("displayName max 50 characters");
  }

  return { displayName: cleanName, bio: cleanBio,
    updatedAt: Date.now() };
}

function formatProfile(profile) {
  if (!profile.bio) {
    return profile.displayName;
  }
  return profile.displayName + " - " + profile.bio;
}

module.exports = { createProfile, formatProfile };
EOF

```

```

npm test
git add src/profile.js
git commit -m "feat: add profile formatter"
git push -u origin feature/APP-104-profile-formatter

```

Naskah role-play

#	Akun	Aksi
1	B + C	Branch + coding + push PR masing-masing (bersamaan)
2	B <-> C	Review silang: B me-review PR C, C me-review PR B
3	A	Approve + squash merge PR yang siap duluan (misal APP-103)
4	C	PR-nya tertinggal: fetch + merge origin/main, resolve conflict, test, push
5	A	Review ulang, approve, squash merge APP-104

#	Akun	Aksi
6	B, C	Sync main, hapus branch

WEB UI – review silang:

1. PR APP-103: Akun C buka "Files changed" -> "Review changes" -> Approve (atau Request changes bila ada temuan)
2. PR APP-104: Akun B melakukan hal yang sama
3. Akun A: "Squash and merge" APP-103 duluan
4. Halaman PR APP-104 langsung berubah: "This branch has conflicts that must be resolved" -> C ke terminal

Konflik alami yang dialami C

```
# Akun C, setelah APP-103 masuk main duluan
git fetch origin
git merge origin/main
```

```
<<<<<< HEAD
module.exports = { createProfile, formatProfile };
=====
module.exports = { createProfile };
>>>>>> origin/main
```

Resolusi: pertahankan maksud KEDUA sisi – badan createProfile otomatis memakai versi main (yang sudah punya location, karena C tidak mengubah bagian itu), dan baris exports tetap memuat formatProfile:

```
module.exports = { createProfile, formatProfile };
```

```
npm test
git add src/profile.js
git commit
git push
```

Aturan mode paralel

- Jangan push ke branch orang lain; beri feedback lewat komentar PR.
- Yang merge belakangan wajib integrasi + test ulang.
- Konflik besar berulang = tanda task perlu dipecah atau butuh koordinasi sebelum coding.

BAB 9 – Simulasi 4 Akun: Dua Reviewer

Mode paling mendekati perusahaan: satu author, dua reviewer wajib, dan maintainer yang mengeksekusi merge. Titik awal: main sudah berisi APP-101 s.d. APP-104.

Role

Akun	Role
A	Tech Lead / Maintainer – aturan, keputusan akhir, merge
B	Reviewer 1 (senior dev)
C	Reviewer 2 (senior dev)
D	Developer – author PR

Setup tambahan (Akun A)

- Require approvals: 2.
- Aktifkan "dismiss stale approvals": push baru menggugurkan approval lama.
- CODEOWNERS agar B dan C otomatis diminta review:

```
cat > .github/CODEOWNERS << 'EOF'
src/  @akun-b @akun-c
EOF
```

Commit file CODEOWNERS ke main (boleh lewat PR kilat oleh A). Setelah itu setiap PR yang menyentuh src/ otomatis meminta review B dan C – panel Reviewers terisi sendiri.

WEB UI – naikkan aturan (Akun A):

1. Settings -> Branches -> edit rule/ruleset main
2. Required approvals: 2
3. Centang "Dismiss stale pull request approvals when new commits are pushed"
4. Save changes

Ticket APP-105 dan implementasi Akun D

APP-105 – Fix empty bio handling

- formatProfile("Raka", bio spasi saja) tidak boleh menghasilkan "Raka – "
- bio berisi spasi saja dianggap kosong

Karena createProfile sudah men-trim bio (Bab 4), bio " " tersimpan sebagai "" – tetapi formatProfile milik C (Bab 8) belum punya test untuk kasus itu. Task D: tambahkan test formatter + export yang

kurang, tanpa mengubah behavior lain.

```
# Akun D - di folder team-work-d
git switch main && git pull origin main
git switch -c fix/APP-105-empty-bio
git branch --show-current # fix/APP-105-empty-bio
```

```
cat >> test/profile.test.js << 'EOF'

const { formatProfile } = require("../src/profile");

test("formats name - bio", () => {
  const p = createProfile("Raka", "backend dev");
  assert.equal(formatProfile(p), "Raka - backend dev");
});

test("whitespace-only bio formats as name only", () => {
  const p = createProfile("Raka", " ");
  assert.equal(formatProfile(p), "Raka");
});
EOF
```

Catatan: blok di atas memakai >> (append) karena kita hanya MENAMBAH test di akhir file – file lain tidak berubah.

```
npm test
git add test/profile.test.js
git commit -m "test: cover empty bio formatting"
git push -u origin fix/APP-105-empty-bio
```

Naskah role-play

#	Akun	Aksi
1	A	Buat ticket APP-105; assign ke D
2	D	Branch, test baru, push, buka PR
3	B	Review: REQUEST CHANGES (minta test tambahan)
4	C	Review: APPROVE – dua reviewer boleh beda pendapat
5	D	Perbaiki di branch yang sama, push
6	–	Approval C otomatis gugur (stale) karena ada commit baru
7	B	Re-review: APPROVE
8	C	Re-review: APPROVE (2/2 lengkap)

#	Akun	Aksi
9	A	Cek checks + conversations, lalu squash merge
10	D	Sync main, hapus branch

Checks [PASS] ci / test
Reviews [PASS] 2/2 approvals (B, C)
Status READY TO MERGE

WEB UI yang terlihat di PR APP-105:
- Setelah D push perbaikan (langkah 5), centang approve C di panel Reviewers berubah abu-abu (stale) otomatis
- Tombol merge terkunci: "At least 2 approving reviews are required" sampai B dan C approve ulang
- Setelah 2/2: tombol "Squash and merge" aktif untuk A

Dinamika dua reviewer

- Reviewer beda pendapat? Diskusikan di thread PR sampai ada keputusan, bukan lewat jalur pribadi. Tech Lead jadi pemutus bila buntu.
- Author tidak pernah meng-approve PR-nya sendiri.
- 2 approval bukan auto-merge – merge tetap keputusan maintainer.
- Commit baru = review ulang. Itu sebabnya PR kecil lebih cepat masuk.

BAB 10 – Cherry-pick

Apa itu

Cherry-pick menyalin SATU commit tertentu ke branch lain sebagai commit baru (SHA berbeda, isi perubahan sama). Berguna saat kamu butuh satu perbaikan tanpa membawa seluruh branch.

Kapan dipakai

- Membawa hotfix dari main ke release branch (kasus paling umum).
- Menyelamatkan commit yang terlanjur dibuat di branch yang salah.
- Mengambil sebagian kecil pekerjaan tanpa merge semuanya.

Command inti

```
git log --oneline main          # cari SHA commit target
git switch release/1.0
git cherry-pick -x <sha>        # -x catat asal commit
git cherry-pick -x <sha1> <sha2> # beberapa sekaligus
git cherry-pick --continue      # setelah resolve conflict
git cherry-pick --abort         # batalkan
```

Simulasi: release/1.0 lahir dari tag v1.0.0

Di Bab 7 kamu membuat tag v1.0.0 SEBELUM hotfix APP-102 masuk. Pelanggan versi 1.0 butuh hotfix itu, tapi tidak boleh menerima APP-103/104/105. Solusi: buat release/1.0 dari tag, lalu cherry-pick HANYA commit hotfix.

```
# Akun A: release branch lahir dari TAG, bukan dari main
git switch main && git pull origin main
git switch -c release/1.0 v1.0.0
git push -u origin release/1.0

# buktikan: release/1.0 belum punya APP-102
git log --oneline -3
grep MAX_NAME src/profile.js # (tidak ada hasil)
```

```
# Developer membawa hotfix APP-102 dari main
git switch main && git pull origin main
git log --oneline | head -8
# cari SHA squash commit Bab 7:
# "fix: limit profile display name length (#15)"

git switch release/1.0
git cherry-pick -x <sha-hotfix>
grep MAX_NAME src/profile.js # sekarang ADA
npm test
git push origin release/1.0
# atau lewat PR bila release branch juga protected
```

```
WEB UI – bila release/1.0 juga protected (opsional):
1. Kerjakan pick di branch sendiri:
   git switch -c backport/APP-102   (dari release/1.0)
   lalu push branch itu
2. "New pull request" -> base: release/1.0 (BUKAN main!)
   <- compare: backport/APP-102
3. Review -> approve -> merge seperti biasa
```

Cherry-pick masuk mulus karena isi release/1.0 (= v1.0.0) sama persis dengan keadaan main saat APP-102 dibuat. Kalau ada perubahan lain di antaranya, conflict bisa muncul – kamu akan memicunya dengan sengaja di Bab 11 Case 3.

Aturan keselamatan

- Cherry-pick = duplikasi commit; bukan pengganti merge untuk pekerjaan besar.
- Selalu pakai -x agar jejak asal commit tercatat.
- Test ulang setelah pick – konteks branch target bisa berbeda.
- Rantai cherry-pick panjang antar branch aktif = resep konflik; pertimbangkan merge.

BAB 11 – Error Handling dan Recovery

Prasyarat: Bab 3-10 selesai, jadi main berisi APP-101 s.d. APP-105 dan release/1.0 sudah ada. Setiap case di bab ini kamu PICU SENDIRI dengan command yang diberikan, lalu perbaiki. Semua eksperimen memakai branch latihan/* atau di-revert, sehingga repo tim tetap bersih. Aturan emas saat panik: BERHENTI, jalankan git status, BACA pesannya – hampir semua operasi punya tombol mundur (--abort), dan hampir semua commit bisa diselamatkan lewat reflog.

Case 1 – Merge conflict

```
# picu: dua branch mengubah baris pemisah formatProfile (Bab 8)
git switch main && git pull origin main
git switch -c latihan/merge-a
# edit src/profile.js, baris return terakhir formatProfile:
#   return profile.displayName + " | " + profile.bio;
git commit -am "chore: separator pipe"

git switch main
git switch -c latihan/merge-b
# edit baris yang sama menjadi:
#   return profile.displayName + " * " + profile.bio;
git commit -am "chore: separator star"

git merge latihan/merge-a          # BOOM
```

```
Auto-merging src/profile.js
CONFLICT (content): Merge conflict in src/profile.js
Automatic merge failed; fix conflicts and then commit
the result.
```

```
<<<<<<< HEAD
    return profile.displayName + " * " + profile.bio;
=====
    return profile.displayName + " | " + profile.bio;
>>>>>>> latihan/merge-a
```

```
# putuskan satu maksud (misal " | "), hapus marker, lalu:
npm test                                # merah? test D di Bab 9
                                         # menuntut " - ": pelajaran!
# kembalikan separator ke " - " agar test hijau, atau sadari
# bahwa mengubah format = mengubah test. utk latihan: " - ".
git add src/profile.js
git commit                              # merge commit
git log --oneline --graph -4            # lihat bentuk merge

# bereskan latihan:
git switch main
git branch -D latihan/merge-a latihan/merge-b
```

Case 2 – Rebase conflict (buktikan arah HEAD terbalik)

```

git switch main && git pull origin main
git switch -c latihan/rebase-x
# edit separator formatProfile menjadi " ~ "
git commit -am "chore: separator tilde"

# simulasikan main maju (cukup lokal, JANGAN push):
git switch main
# edit separator menjadi " / "
git commit -am "chore: separator slash"

git switch latihan/rebase-x
git rebase main # BOOM

```

```

<<<<<< HEAD
    return profile.displayName + " / " + profile.bio;
=====
    return profile.displayName + " ~ " + profile.bio;
>>>>>> (chore: separator tilde)

```

Bandingkan dengan Case 1: sekarang HEAD justru versi MAIN (" / "), dan sisi bawah adalah commit-mu (" ~ ") yang sedang direplay – kebalikan dari merge. Makna --ours/--theirs pun ikut terbalik.

```

# selesaikan (pilih maksudmu), lalu:
git add src/profile.js
git rebase --continue
# conflict bisa muncul lagi untuk commit berikutnya – ulangi;
git rebase --skip # bila commit menjadi kosong
git rebase --abort # menyerah, kembali seperti semula

# tip: Git bisa mengingat resolusi yang berulang
git config rerere.enabled true

# bereskan: main lokal ikut kita ubah tadi – kembalikan:
git switch main
git reset --hard origin/main # aman: belum dipush
git branch -D latihan/rebase-x

```

Case 3 – Cherry-pick conflict

```

# picu: ubah baris MAX_NAME (Bab 7) di dua tempat
git switch release/1.0 && git pull origin release/1.0
# edit src/profile.js -> const MAX_NAME = 60;
git commit -am "chore: raise limit to 60 (release only)"

git switch main && git pull origin main
# edit src/profile.js -> const MAX_NAME = 40;
git commit -am "chore: tighten limit to 40"
git log --oneline -1 # catat SHA, misal ab12cd3

git switch release/1.0
git cherry-pick -x ab12cd3 # BOOM – conflict yang kita mau

```



```
error: could not apply ab12cd3... chore: tighten limit to 40
hint: After resolving the conflicts, mark them with
hint: "git add/rm <paths>", then run
hint: "git cherry-pick --continue"
```

```
<<<<<< HEAD
const MAX_NAME = 60;
=====
const MAX_NAME = 40;
>>>>>> ab12cd3 (chore: tighten limit to 40)
```

Pada cherry-pick, HEAD = branch TARGET (release/1.0); sisi bawah = commit yang sedang dibawa. Putuskan sesuai kebutuhan branch target, hapus marker, pastikan file tetap utuh, lalu:

```
git status                # lihat file yang unmerged
# edit file: sisakan const MAX_NAME = 40; tanpa marker
git add src/profile.js
git cherry-pick --continue

# ingin membatalkan seluruh cherry-pick? kapan saja:
git cherry-pick --abort
```

Bonus pelajaran: npm test sekarang MERAH, karena test boundary Bab 7 masih menuntut angka 50. Mengubah behavior = wajib mengubah test. Setelah puas bereksperimen, bereskan dengan revert:

```
git revert --no-edit HEAD~1 HEAD    # di release/1.0 (2 commit)
git switch main && git revert --no-edit HEAD
npm test                            # hijau kembali
```

Case 4 – Cherry-pick kosong / duplikat

```
# picu: bawa commit yang perubahannya SUDAH ada di target
git cherry-pick -x ab12cd3
```

```
The previous cherry-pick is now empty, possibly due to
conflict resolution. If you wish to commit it anyway, use:
  git commit --allow-empty
Otherwise, please use 'git cherry-pick --skip'
```

```
git cherry-pick --skip    # lewati, karena memang tidak perlu
```

Case 5 – Push ditolak: non-fast-forward (race dua akun)

```
# Akun C - team-work-c: buat branch latihan bersama
git switch main && git pull origin main
git switch -c latihan/push-race
git push -u origin latihan/push-race
```

```
# Akun B - team-work-b: ikut ke branch yang sama
git fetch origin
git switch latihan/push-race
echo "catatan dari B" >> README.md
git commit -am "docs: catatan B"
git push                                # B masuk duluan
```

```
# Akun C - commit TANPA pull dulu:
echo "catatan dari C" >> README.md
git commit -am "docs: catatan C"
git push                                # BOOM
```

```
! [rejected] latihan/push-race -> latihan/push-race
(fetch first)
error: failed to push some refs
hint: Updates were rejected because the remote contains work
hint: that you do not have locally.
```

```
# Akun C:
git pull --rebase origin latihan/push-race
# (kedua catatan menyentuh baris terakhir README -> bila
# conflict, selesaikan persis seperti Case 2)
git push                                # sekarang diterima
```

```
# bereskan:
git switch main
git branch -D latihan/push-race
git push origin --delete latihan/push-race
```

Case 6 - --force-with-lease ditolak (stale info)

Lanjutan Case 5: misal C memilih rebase lokal lalu force push, tetapi B sempat push lagi SETELAH fetch terakhir C:

```
! [rejected] latihan/push-race -> latihan/push-race
(stale info)
```

```
# artinya: remote berubah sejak fetch terakhirmu - BAGUS,
# kamu baru saja diselamatkan dari menimpa kerja orang.
git fetch origin
git log --oneline origin/latihan/push-race -3    # cek dulu
# integrasikan/koordinasikan, baru:
git push --force-with-lease --force-if-includes
```

Case 7 - Pindah branch ditolak: stash

```
# picu: edit tanpa commit, lalu pindah ke branch yang isinya
# beda (main punya APP-103/104, release/1.0 tidak)
git switch main
echo "// coretan latihan" >> src/profile.js
git switch release/1.0
```

```
error: Your local changes to the following files would be
overwritten by checkout: src/profile.js
```

```
git stash                # titipkan dulu
git switch release/1.0   # sekarang bisa
git switch -             # kembali ke main
git stash pop            # ambil lagi coretanmu
git restore src/profile.js # buang coretan latihan
```

Case 8 – Salah pesan / lupa file: amend

```
git switch main && git switch -c latihan/amend
echo "## Catatan rilis" >> README.md
git add README.md
git commit -m "docs: tambah catatan"      # typo disengaja

echo "- v1.0.0" >> README.md              # eh, ada yang lupa
git add README.md
git commit --amend -m "docs: tambah catatan rilis"
git log --oneline -1                     # satu commit, rapi

# JANGAN amend commit yang sudah dipush ke branch bersama
git switch main && git branch -D latihan/amend
```

Case 9 – reset, reflog, revert, detached HEAD (satu alur)

```
git switch main && git switch -c latihan/undo
echo "percobaan-1" >> README.md && git commit -am "chore: p1"
echo "percobaan-2" >> README.md && git commit -am "chore: p2"

# (a) batalkan commit terakhir, perubahan tetap ada:
git reset --soft HEAD~1 && git status
git commit -m "chore: p2 lagi"

# (b) buang total commit terakhir:
git reset --hard HEAD~1
git log --oneline -2                # p2 "hilang"
```

```
# (c) selamatkan yang "hilang" dengan reflog:
git reflog -5
# cari baris sebelum reset, misal HEAD@{1}, lalu:
git switch -c latihan/rescue HEAD@{1}
git log --oneline -1                # p2 kembali!
```

```
# (d) revert: pembalik yang aman utk commit yang SUDAH dipush
git revert --no-edit HEAD
git log --oneline -3          # commit + pembaliknya
```

```
# (e) detached HEAD: melompat ke SHA lama
git checkout HEAD~1
# "You are in 'detached HEAD' state..."
git switch -c latihan/detached  # selamatkan bila perlu

# bereskan semua:
git switch main
git branch -D latihan/undo latihan/rescue latihan/detached
```

Situasi	Command	Efek
Commit lokal, BELUM push	git reset --soft HEAD~1	Commit batal, perubahan tetap ada (staged)
Commit lokal, buang total	git reset --hard HEAD~1	Commit dan perubahan hilang (hati-hati)
SUDAH dipush / branch bersama	git revert <sha>	Commit baru pembalik; history tetap aman

Case 10 – File nyasar ikut ke-commit

```
git switch main && git switch -c latihan/nyasar
mkdir -p node_modules/fake
echo "{}" > node_modules/fake/package.json
git add -f node_modules      # -f menembus .gitignore: NAKAL
git commit -m "oops: node_modules ikut ke-commit"

git rm -r --cached node_modules
git commit -m "chore: remove node_modules from repo"
git switch main && git branch -D latihan/nyasar
rm -rf node_modules
```

Kalau yang bocor adalah SECRET/token dan sudah TER-PUSH: anggap sudah bocor – revoke/rotate credential sekarang juga, baru bersihkan history sesuai kebijakan tim.

Toolbox pengendali conflict

```
git status          # peta situasi
git diff --name-only --diff-filter=U  # file masih conflict
git checkout --ours  <file>          # versi sisimu (merge)
git checkout --theirs <file>         # versi sisi lain
# ingat: saat REBASE makna ours/theirs terbalik (Case 2)

git merge --abort
git rebase --abort
git cherry-pick --abort
```

Ringkasan penyelamat

Panik karena	Penyelamat
Conflict kacau (merge/rebase/pick)	<code>--abort</code> masing-masing operasi
Commit hilang / reset kebablasan	<code>git reflog + switch -c rescue</code>
Push ditolak	<code>git pull --rebase</code> , jangan <code>force</code>
Salah branch / salah commit	<code>switch -c</code> , <code>cherry-pick</code> , <code>revert</code>
Perubahan belum siap commit	<code>git stash / stash pop</code>
Tidak tahu sedang terjadi apa	<code>git status + git log --graph</code>

BAB 12 – Penutup

Workflow ringkas

```
MULAI : git switch main -> git pull -> switch -c feature/T-x
CEK   : git branch --show-current SEBELUM commit apa pun
KERJA : edit file UTUH -> npm test -> status/diff -> add -> commit
KIRIM : git push -u origin <branch> -> buka PR -> self-review
REVIEW : request changes? perbaiki branch yang SAMA -> push
MERGE : checks hijau + approval (mode 4: dua) -> squash merge
SELESAI: sync main -> hapus branch
        (perlu di release? cherry-pick -x)
```

Kesalahan yang harus dihindari

Kesalahan	Perbaikan
Coding langsung di main	Branch per task; cek <code>--show-current</code>
Menyalin potongan code	Selalu tulis file versi UTUH
<code>git add .</code> tanpa lihat diff	Stage hanya file relevan
PR baru saat request changes	Perbaiki branch yang sama
Asal pilih sisi conflict	Pahami dua maksud, gabungkan, test
Force push polos	<code>--force-with-lease</code> sesuai policy
Merge saat CI merah	Perbaiki checks dulu
Approve PR sendiri	Review silang / tunggu reviewer
Cherry-pick tanpa <code>-x</code>	Jejak asal commit hilang

Kriteria lulus

- [] SSH multi-akun jalan: `ssh -T <alias>` menyapa akun yang benar
- [] Hafal alur Web UI: issue -> PR -> review -> squash merge -> release
- [] Satu siklus penuh APP-101: branch -> PR -> review -> squash merge
- [] Menangani request changes tanpa PR baru
- [] Conflict updatedAt (Bab 6) selesai dan terbawa sampai akhir
- [] Tag v1.0.0 dibuat sebelum hotfix, release/1.0 lahir dari tag
- [] Mode 3 akun: dua PR paralel + review silang + konflik alami selesai
- [] Mode 4 akun: 2 approval, paham stale review, merge oleh maintainer

- [] Cherry-pick hotfix ke release branch dengan -x
- [] Minimal 5 case error di Bab 11 dipicu sendiri dan diperbaiki tanpa melihat buku

Kalau semua tercentang dan kamu bisa menjelaskan KENAPA setiap langkah dilakukan – bukan sekadar hafal – kamu siap masuk ke tim software sungguhan. Ulangi simulasi dengan rotasi role sampai semua posisi pernah kamu rasakan.